

xrpl.js developer experience audit

Prepared for Ripple DevRel by aha · 22 September 2026

Baseline: published `xrpl@5.3.0`, TypeScript 5.9.3. **Focus:** an experienced TypeScript developer learning XRPL through the editor and public examples.

Executive assessment

xrpl.js has useful building blocks for a good TypeScript experience. Transaction discriminants already help developers choose required fields, ordinary inferred event handlers have useful payload types, and the confidential-operation recipe uses a helpful discriminated union. The opportunity is to make that guidance continuous throughout an application workflow.

Today, guidance breaks at several important boundaries. Published declarations hide existing inline documentation. Permissive models accept misspelled fields. Preparation, signing and specific queries do not consistently preserve what the library already knows. Some TypeScript examples bypass these protections or fail to teach reliable outcome handling.

These are recurring design problems with effects across multiple workflows. A sustained engagement can address their shared causes, maintain the examples that teach the API, and prevent regressions in the package developers actually install.

This audit combines a complete public export inventory, a broad review across 18 API families, selected compiler and runtime reproductions, and paired public-example improvements. It is an expert assessment, not a developer study or exhaustive protocol correctness audit. No developer-time savings or production losses are inferred from these tests.

Coverage and evidence

Area	Work completed	What the result means
Public package	447 named root exports, including 137 runtime values; 69 qualified namespace members; public Client and Wallet members	Complete inventory of this defined surface, not exhaustive execution of every member
API families	Explicit review depth for all 18 families	Representative source, type and behavior checks, with remaining uncertainty recorded
Editor experience	Strict compiler probes and actual TypeScript language-service hover queries	Reproducible accepted programs, rejected programs, inferred types and available guidance
Public examples	All seven tracked TypeScript families reviewed	Four selected journeys improved; legacy samples remain clearly identified
Ledger workflows	Payment, MPT issuance/update/inspection, and AMM setup/create/query on an isolated ledger	Real transactions in the recorded test configuration; no claim about every public-network amendment state
Earlier work	107 historical claims reconciled to 27 open aha fix proposals	Historical claims, current reproductions and proposed fixes remain separate

The current report groups observations into **12 SDK topics and six portal themes**. These are reporting units of different evidence strengths, including a design opportunity and source-only observations. They are not 18 demonstrated runtime bugs, and they must not be added to the earlier 107 claims.

The installed 5.3.0 SDK source matches the earlier local SDK source at `f79b82c906a6379b51bc08cb64eb1bef2557d77e`. The earlier audit nevertheless used a local 5.2.0 build. Reproduction against the published artifact matters because package contents and editor behavior are part of the product.

Findings that should lead the discussion

1. Useful documentation disappears before it reaches the editor

None of the 447 inventoried root exports exposes JSDoc prose in the installed declaration graph. The shipped source has descriptions for 344 of those symbols. Actual hovers for the `Client` constructor, `Client.request`, `Wallet.generate` and `xrpToDrops` return signatures without documentation. Inherited `EventEmitter` documentation is an exception.

The build removes comments. Preserving declaration comments restores guidance in the tested hovers. This is an immediate, broad improvement, but the restored prose needs review: existing source examples include a nonexistent `signTransaction` call and an inconsistent `submitAndWait` walkthrough. Package-consumer hover checks and compilation of documentation examples should become release checks. **Evidence: DX-01, DX-12.**

2. Types should prevent mistakes and carry intent between steps

On 5.3.0, `DestinationTag` in a `Payment` and `ledger_idx` in an account query compile. `Omit<Payment, 'Account'>` can lose the other required-field checks. These locally knowable mistakes should be caught before a request is sent.

The opposite problem occurs with valid code. `autofill` returns the original inferred shape even after populating `Fee` and `LastLedgerSequence`. Signing turns a typed transaction into an ordinary string, so subsequent submission loses its transaction-specific metadata. A typed issuance object submitted directly already retains its metadata type; the loss demonstrated here occurs across the signed-blob boundary.

Likewise, an MPT-specific ledger selector still returns a broad ledger-entry union. A developer knows what was requested but must explain it to TypeScript again. The prototype makes literal selectors inform the response while preserving safe unions for broad, ambiguous and binary requests. **Evidence: DX-02, DX-03, DX-04.**

3. Broader surface checks reveal additional work

Topic	Reproduced observation	Recommended direction
Events	An explicitly incompatible on callback is accepted; inherited once loses its payload to any	One checked event-to-argument map across registration methods
Pagination	<code>account_nfts</code> is rejected by one paging helper and accepted by another that immediately fails with its defaults	Consistent typed paging and explicit collection/default semantics
Preconditions	A tx query can omit both identifiers or provide both; unsigned submission can omit its required wallet	Model locally knowable alternatives and required options
Recovery	Narrowing to <code>RippledError</code> still leaves RPC error data unknown	Checked accessors for known error fields; validated decoding paths
Balance utilities	A synthetic public-MPT balance change produces no balance-change result	State the existing scope and add verified public-MPT support
Exact amounts	A very large drops-to-XRP-to-drops conversion changes the value by one drop	Offer exact string/decimal paths with a migration plan

The precision example is 9007199254740991 drops becoming 9007199254740992 after the round trip. It demonstrates a representation limitation, not an observed transfer or loss. The MPT balance finding uses a synthetic fixture, not a captured transaction. Full details and limits are in DX-05 through DX-11.

The developer journey before and after

The walkthrough uses three clearly labeled states: **existing tracked examples**, **corrected examples that work with published 5.3.0**, and **paired examples using the unreleased aha prototype**. This distinction separates example improvements available immediately from SDK improvements that require a release.

Get Started: learn from inputs and inferred responses

The proposed main walkthrough passes request and transaction literals directly to the client. `command: 'account_info'` and `TransactionType: 'Payment'` guide completion and inference, without importing the corresponding types, adding `satisfies`, or casting. Actual editor checks confirm `Payment` fields are offered and missing required fields or misspelled optional fields are rejected.

`submitAndWait` prepares and validates during signing, so the first-payment example needs no separate `validate()` call. The prototype guarantees parsed metadata; the application reads `TransactionResult` directly. The `tessUCCESS` check stays, subscription errors are observed, and `finally` closes the connection. The published 5.3.0 workaround version is retained separately for comparison. **Validated means included in a validated ledger; it does not mean the requested business operation succeeded.**

MPT issuance: the selected object should guide its answer

Both versions create an issuance, check success, require the returned issuance ID, query metadata, update it and confirm the result. With the prototype, `mpt_issuance` selects `MPTokenIssuance`, so an extra ledger-kind check disappears. Optional metadata and issuance-ID checks remain because they describe real domain possibilities.

The mutable-MPT walkthrough requires `DynamicMPT` support. The audit's isolated ledger explicitly enables the required amendments. Network availability is a runtime condition, not something stronger TypeScript can establish.

Signing and AMM: carry useful guarantees without hiding failures

The Send XRP pair demonstrates `autofill`, signing and confirmation. The current-release example needs an explicit transaction generic to expose prepared fields. The prototype infers populated fields and keeps the transaction type through its signed blob.

Create AMM adds a focused, fully typed setup/create/query walkthrough beside the original broad example. Each setup transaction is checked independently. The original advanced sample remains available and is not represented as remediated; it still needs work on any, repeated result handling and failure propagation.

Across the four paired workflows, the prototype removes five parsed-metadata guards, two MPT ledger-kind comparisons and one `autofill` generic argument. Getting Started additionally removes two redundant `satisfies` clauses and a separate validation call. All transaction-success checks remain. These are scoped source changes, not a usability score or measured time saving.

Prototype and compatibility

The prototype reuses selected work from existing aha proposals for ledger models, stricter inputs, query narrowing and transaction lifecycle types. Integration added regression checks for broad and union inputs, corrected normalized flag types, retained declaration comments, and tightened direct submission calls. It is an evaluation branch, not a release recommendation.

Independent review identified two changes that should not be accepted merely because their tests passed: a proposed simulation return type described pre-normalized flags, and proposed submission handling treated potentially nonfinal results as terminal. The demonstration keeps existing simulation/finality behavior and limits its new guarantees to behavior backed by checks. The review record identifies the source and decisions.

The stricter input types, richer return types and const type parameters require compatibility review. Plan migration guidance, an explicit path for forward-compatible requests, and a declared minimum TypeScript version. Restoring comments also requires maintaining their correctness. Do not ship the combined prototype wholesale without the integration and release work described below.

Scope remains important even when selected probes pass: `direct wallet.sign` calls can still accept extra fields, and a signed-blob type brand carries information without validating arbitrary external bytes. The prototype's reliable-confirmation lookup explicitly uses API v2 to match its documented response shape. These contracts need to be included in compatibility review.

Implementation roadmap

Phase	Deliverable	Acceptance evidence
Restore editor guidance	Correct and preserve declaration documentation	Packed-package hover checks; compiled inline examples
Make workflows coherent	Reviewed strict input, preparation/signing and query-response contracts	Positive and negative consumer tests; broad/union safety checks; migration examples
Maintain public teaching paths	Current-release and candidate examples, aligned tutorials and reliable outcome checks	Strict compilation, isolated-ledger execution and rendered docs
Address broader gaps	Events, paging, error recovery, exact values and MPT helper scope	Targeted real or captured fixtures; documented limitations and compatibility
Sustain quality	Package-consumer and example checks in release workflows	Reproducible evidence on each proposed release

The first engagement should finish and release a bounded set of agreed improvements, with maintainers reviewing the contracts and DevRel reviewing the teaching journey. Effort and sequencing should be agreed after technical triage rather than inferred from issue counts.

For the longer partnership, aha can take recurring responsibility for discovery, implementation, public examples, documentation and regression prevention. A shared backlog should identify developer impact, evidence, owner, dependency and acceptance criteria. The next audit phase applies the method to the Rust crate and CLI, including command discovery, help text, errors and scripting behavior.

Evidence, handoff and limits

The accompanying findings register links each group to reproducible evidence and earlier work. The API coverage matrix distinguishes inventory from deeper investigation. Baseline and candidate compiler output, local runtime results, example diffs, test logs, source provenance and review notes are retained with the audit.

All code and publication targets are aha forks. The docs preview runs locally at port 4400. The initial four tutorials were visually checked. The revised Getting Started page passes compilation and source-reference checks; refreshed browser review remains outstanding. A shareable hosted Realm preview needs compatible hosting configuration; that optional deployment is not part of the completed evidence.

The audit does not claim a cryptographic review, complete amendment/transaction coverage, browser runtime execution, observed novice success rates, or that every historical claim was reverified. The evidence is sufficient to prioritize and demonstrate a concrete improvement program while making the remaining work visible.

Companion files: [finding register](#), [API coverage](#), [portal review](#), [paired example excerpts](#), [validation summary](#), and the editable walkthrough deck.